

Advanced Machine Learning

T.A Merit Samir



Lab 3

1. Clustering
2. K-Means Clustering



clustering

Clustering is a machine learning technique that **groups similar data points together** based on specific features or characteristics.

It's a form of unsupervised learning, meaning it's used on data without predefined labels.

Clustering algorithms aim to maximize the similarity of data points within clusters and the dissimilarity between clusters.

It's widely used in various applications like customer segmentation, image segmentation, anomaly detection, and document clustering.



K-Means Clustering

- **How it works:** Divides the dataset into k clusters by finding k centroids and assigning each data point to the closest centroid.
- **Application:** Suitable for relatively spherical clusters; commonly used in customer segmentation, and market segmentation.
- **Key Parameters:** `n_clusters` (number of clusters), `init` (initialization method, e.g., 'k-means++').



K-Means Cont.

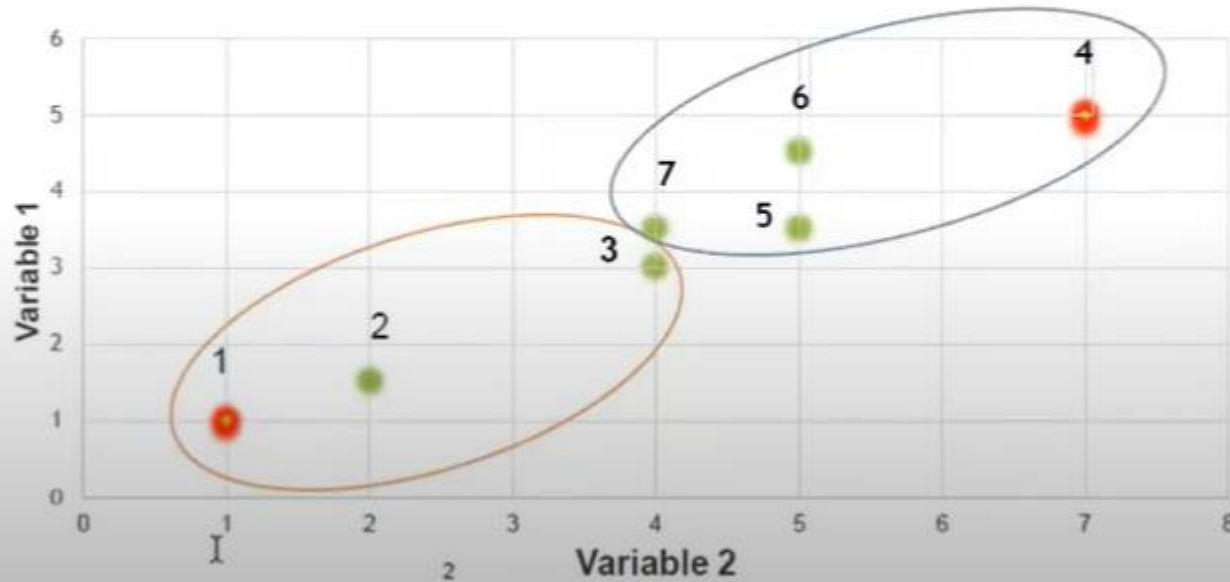
A Simple example k-means (using K=2)

Individual	Variable 1	Variable 2
1	1	1
2	1.5	2
3	3	4
4	5	7
5	3.5	5
6	4.5	5
7	3.5	4.5



K-Means Cont.

K= 2



K-Means Cont.

Step 1:

Initialization: Randomly we choose following two centroids ($k=2$) for two clusters. In this case the 2 centroid are: $m1=(1.0, 1.0)$ and $m2=(5.0, 7.0)$.

	Individual	Mean Vector
Group 1	1	(1.0, 1.0)
Group 2	4	(5.0, 7.0)

K-Means Cont.

Step 2:

	Centroid 1	Centroid 2
1	$f(1 - 1)^2 + (1 - 1)^2 = 0$	$f(5 - 1)^2 + (7 - 1)^2 = 7.21$
2	$f(1 - 1.5)^2 + (1 - 2)^2 = 1.12$	$f(5 - 1.5)^2 + (7 - 2)^2 = 6.10$
3	$f(1 - 3)^2 + (1 - 4)^2 = 3.61$	$f(5 - 3)^2 + (7 - 4)^2 = 3.61$
4	$f(1 - 5)^2 + (1 - 7)^2 = 7.21$	$f(5 - 5)^2 + (7 - 7)^2 = 0$
5	$f(1 - 3.5)^2 + (1 - 5)^2 = 4.72$	$f(5 - 3.5)^2 + (7 - 5)^2 = 2.5$
6	$f(1 - 4.5)^2 + (1 - 5)^2 = 5.31$	$f(5 - 4.5)^2 + (7 - 5)^2 = 2.06$
7	$f(1 - 3.5)^2 + (1 - 4.5)^2 = 4.30$	$f(5 - 3.5)^2 + (7 - 4.5)^2 = 2.92$

K-Means Cont.

Step 2:

- Thus, we obtain two clusters containing:
{1,2,3} and {4,5,6,7}.
- Their new centroids are:

$$\text{Group 1} = \left(\frac{1+1.5+3}{3} \right), \left(\frac{1+2+4}{3} \right) = (1.83, 2.33)$$

$$\text{Group 2} = \left(\frac{5+3.5+4.5+3.5}{4} \right), \left(\frac{7+5+5+4.5}{4} \right) = (4.12, 5.38)$$

K-Means Cont.

Step 3:

	Centroid 1	Centroid 2
1	$f(1.83 - 1)^2 + (2.33 - 1)^2 = 1.57$	$f(4.12 - 1)^2 + (5.38 - 1)^2 = 5.38$
2	$f(1.83 - 1.5)^2 + (2.33 - 2)^2 = 0.47$	$f(4.12 - 1.5)^2 + (5.38 - 2)^2 = 4.29$
3	$f(1.83 - 3)^2 + (2.33 - 4)^2 = 2.04$	$f(4.12 - 3)^2 + (5.38 - 4)^2 = 1.78$
4	$f(1.83 - 5)^2 + (2.33 - 7)^2 = 5.64$	$f(4.12 - 5)^2 + (5.38 - 7)^2 = 1.84$
5	$f(1.83 - 3.5)^2 + (2.33 - 5)^2 = 3.15$	$f(4.12 - 3.5)^2 + (5.38 - 5)^2 = 0.73$
6	$f(1.83 - 4.5)^2 + (2.33 - 5)^2 = 3.78$	$f(4.12 - 4.5)^2 + (5.38 - 5)^2 = 0.54$
7	$f(1.83 - 3.5)^2 + (2.33 - 4.5)^2 = 2.74$	$f(4.12 - 3.5)^2 + (5.38 - 4.5)^2 = 1.08$

K-Means Cont.

Therefore, the new clusters are:

~~{1,2} and {3,4,5,6,7}~~

✓

$$\text{Group 1} = \left(\frac{1+1.5}{2} \right), \left(\frac{1+2}{2} \right) = (1.25, 1.5)$$

$$\text{Group 2} = \left(\frac{3+5+3.5+4.5+3.5}{5} \right), \left(\frac{4+7+5+5+4.5}{5} \right) = (3.9, 5.1)$$

K-Means Cont.

Step 4:

	Centroid 1	Centroid 2
1	$\sqrt{(1.25 - 1)^2 + (1.5 - 1)^2} = 0.58$	$\sqrt{(3.9 - 1)^2 + (5.1 - 1)^2} = 5.02$
2	$\sqrt{(1.25 - 1.5)^2 + (1.5 - 2)^2} = 0.56$	$\sqrt{(3.9 - 1.5)^2 + (5.1 - 2)^2} = 3.92$
3	$\sqrt{(1.25 - 3)^2 + (1.5 - 4)^2} = 3.05$	$\sqrt{(3.9 - 3)^2 + (5.1 - 4)^2} = 1.42$
4	$\sqrt{(1.25 - 5)^2 + (1.5 - 7)^2} = 6.66$	$\sqrt{(3.9 - 5)^2 + (5.1 - 7)^2} = 2.20$
5	$\sqrt{(1.25 - 3.5)^2 + (1.5 - 5)^2} = 4.16$	$\sqrt{(3.9 - 3.5)^2 + (5.1 - 5)^2} = 0.41$
6	$\sqrt{(1.25 - 4.5)^2 + (1.5 - 5)^2} = 4.78$	$\sqrt{(3.9 - 4.5)^2 + (5.1 - 5)^2} = 0.61$
7	$\sqrt{(1.25 - 3.5)^2 + (1.5 - 4.5)^2} = 3.75$	$\sqrt{(3.9 - 3.5)^2 + (5.1 - 4.5)^2} = 0.72$

K-Means Cont.

- ▶ Therefore, there is no change in the cluster.
- ▶ Thus, the algorithm comes to a halt here and final result consist of 2 clusters $\{1,2\}$ and $\{3,4,5,6,7\}$.



K-Means Cont.

Pros and cons

Advantages of k-means

1. Relatively simple to implement.
2. Scales to large data sets.
3. Guarantees convergence.
4. Easily adapts to new examples.

Disadvantages of k-means

1. Choosing k manually.
2. Being dependent on initial values.
3. Scaling with number of dimensions.



K-Means Example

Read
Mall_Customers
dataset

```
CustomerID,Genre,Age,Annual Income (k$),Spending Score (1-100)
0001,Male,19,15,39
0002,Male,21,15,81
0003,Female,20,16,6
0004,Female,23,16,77
0005,Female,31,17,40
0006,Female,22,17,76
0007,Female,35,18,6
0008,Female,23,18,94
0009,Male,64,19,3
0010,Female,30,19,72
0011,Male,67,19,14
0012,Female,35,19,99
0013,Female,58,20,15
0014,Female,24,20,77
0015,Male,37,20,13
0016,Male,22,20,79
0017,Female,35,21,35
0018,Male,20,21,66
0019,Male,52,23,29
0020,Female,35,23,98
0021,Male,35,24,35
0022,Male,25,24,73
0023,Female,46,25,5
0024,Male,31,25,73
0025,Female,54,28,14
0026,Male,29,28,82
0027,Female,45,28,32
0028,Male,35,28,61
0029,Female,40,29,31
0030,Female,23,29,87
0031,Male,60,30,4
0032,Female,21,30,73
0033,Male,53,33,4
0034,Male,18,33,92
0035,Female,49,33,14
0036,Female,21,33,81
```



K-Means Cont.

Importing the libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

[1] ✓ 6.4s

Importing the dataset

```
dataset = pd.read_csv('Mall_Customers.csv')
X = dataset.iloc[:, [3, 4]].values
```

✓ 0.0s

```
X
✓ 0.0s
array([[ 15,  39],
       [ 15,  81],
       [ 16,   6],
       [ 16,  77],
       [ 17,  40],
       [ 17,  76],
       [ 18,   6],
       [ 18,  94],
       [ 19,   3],
       [ 19,  72],
       [ 19,  14],
       [ 19,  99],
       [ 20,  15],
       [ 20,  77],
       [ 20,  13],
       [ 20,  79],
       [ 21,  35],
       [ 21,  66],
       [ 23,  29],
       [ 23,  98],
       [ 24,  35],
       [ 24,  73],
       [ 25,   5],
       [ 25,  73],
       [ 28,  14],
       ..
       [120,  79],
       [126,  28],
       [126,  74],
       [137,  18],
       [137,  83]], dtype=int64)
```



K-means Cont.

Feature Scaling

```
from sklearn.preprocessing import MinMaxScaler
scaler=MinMaxScaler()
scaler.fit(X)
X=scaler.transform(X)
```

✓ 0.0s

```
X
✓ 0.0s
array([[0.          , 0.3877551 ],
       [0.          , 0.81632653],
       [0.00819672, 0.05102041],
       [0.00819672, 0.7755102 ],
       [0.01639344, 0.39795918],
       [0.01639344, 0.76530612],
       [0.02459016, 0.05102041],
       [0.02459016, 0.94897959],
       [0.03278689, 0.02040816],
       [0.03278689, 0.7244898 ],
       [0.03278689, 0.13265306],
       [0.03278689, 1.          ],
       [0.04098361, 0.14285714],
       [0.04098361, 0.7755102 ],
       [0.04098361, 0.12244898],
       [0.04098361, 0.79591837],
       [0.04918033, 0.34693878],
       [0.04918033, 0.66326531],
       [0.06557377, 0.28571429],
       [0.06557377, 0.98979592],
       [0.07377049, 0.34693878],
       [0.07377049, 0.73469388],
       [0.08196721, 0.04081633],
       [0.08196721, 0.73469388],
       [0.1065738 , 0.13265306],
       ...
       [0.86065574, 0.79591837],
       [0.90983607, 0.2755102 ],
       [0.90983607, 0.74489796],
       [1.          , 0.17346939],
       [1.          , 0.83673469]])
```

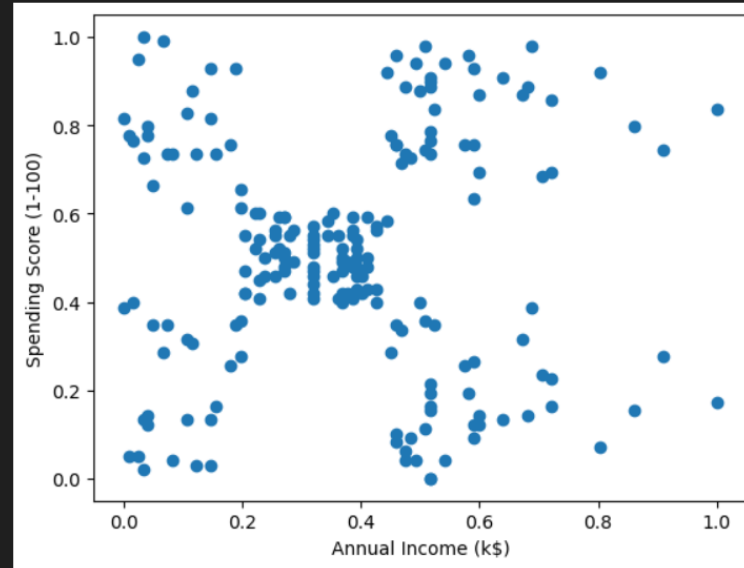


K-means Cont.

```
plt.scatter(X[:,0],X[:,1])  
plt.xlabel("Annual Income (k$)")  
plt.ylabel("Spending Score (1-100)")
```

✓ 0.2s

```
Text(0, 0.5, 'Spending Score (1-100)')
```



K-means Cont.

Using the elbow method to find the optimal number of clusters

```
from sklearn.cluster import KMeans
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters = i, init = 'k-means++', random_state = 42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1, 11), wcss)
plt.title('The Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```

✓ 8.7s



K-means Cont.

This code applies the Elbow Method to determine the optimal number of clusters for the KMeans clustering model.

1.Imports KMeans from scikit-learn: `from sklearn.cluster import Kmeans`

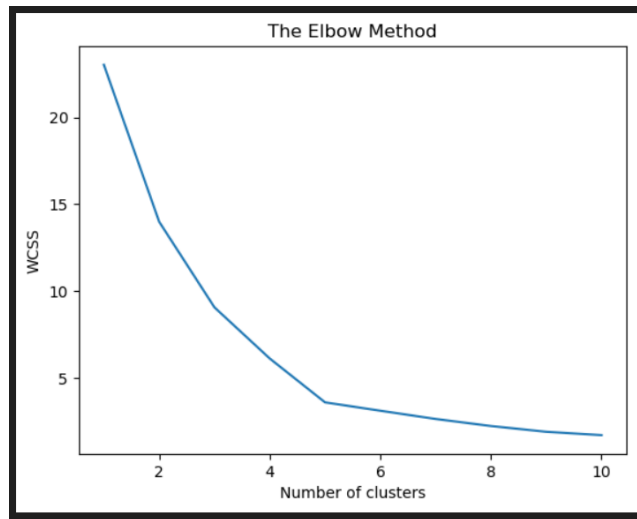
2.Creates a list wcss: This list will store the within-cluster sum of squares (WCSS) for each number of clusters.

3.Iterates over a range of cluster numbers (1 to 10): For each iteration, a KMeans model is trained with `n_clusters = i`, and the model's WCSS (inertia) is added to the wcss list.



K-means Cont.

4.Plots the Elbow Method graph: Using `plt.plot`, it plots the number of clusters (on the x-axis) against WCSS (on the y-axis). The "elbow" point in the plot indicates an optimal number of clusters for the model.



K-means Cont.

Training the K-Means model on the dataset

```
kmeans = KMeans(n_clusters = 5, init = 'k-means++', random_state = 42)  
y_kmeans = kmeans.fit_predict(X)
```

✓ 0.7s

```
kmeans.cluster_centers_
```

✓ 0.0s

```
array([[0.08792846, 0.79962894],  
       [0.6         , 0.16443149],  
       [0.58638083, 0.82783883],  
       [0.33029751, 0.49508692],  
       [0.09265859, 0.20319432]])
```



K-means Cont.

Visualising the clusters

```
plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100, c = 'red', label = 'Cluster 1')
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 100, c = 'blue', label = 'Cluster 2')
plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 100, c = 'green', label = 'Cluster 3')
plt.scatter(X[y_kmeans == 3, 0], X[y_kmeans == 3, 1], s = 100, c = 'cyan', label = 'Cluster 4')
plt.scatter(X[y_kmeans == 4, 0], X[y_kmeans == 4, 1], s = 100, c = 'magenta', label = 'Cluster 5')
plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1], s = 300, c = 'yellow', label = 'Centroids')
plt.title('Clusters of customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()
```

✓ 0.3s



K-means Cont.

In this code, plotting clusters generated by the KMeans algorithm and visualizing each one with a unique color and label.

Cluster Points: Each `plt.scatter` line filters points belonging to a specific cluster and plots them in a different color:

- `X[y_kmeans == i, 0]` and `X[y_kmeans == i, 1]` select the data points in the X dataset that belong to the i-th cluster for plotting.
- The `c` parameter sets each cluster's color, and `label` names each cluster for the legend.

2.Centroids: The centroids of each cluster are plotted in yellow using `kmeans.cluster_centers_`. They're plotted as larger points (`s=300`) to stand out.



K-means Cont.

3.Graph Titles and Labels:

- `plt.title` adds a title to the graph, labeling it as "Clusters of customers."
- `plt.xlabel` and `plt.ylabel` set the x- and y-axis labels to "Annual Income (k\$)" and "Spending Score (1-100)," respectively.

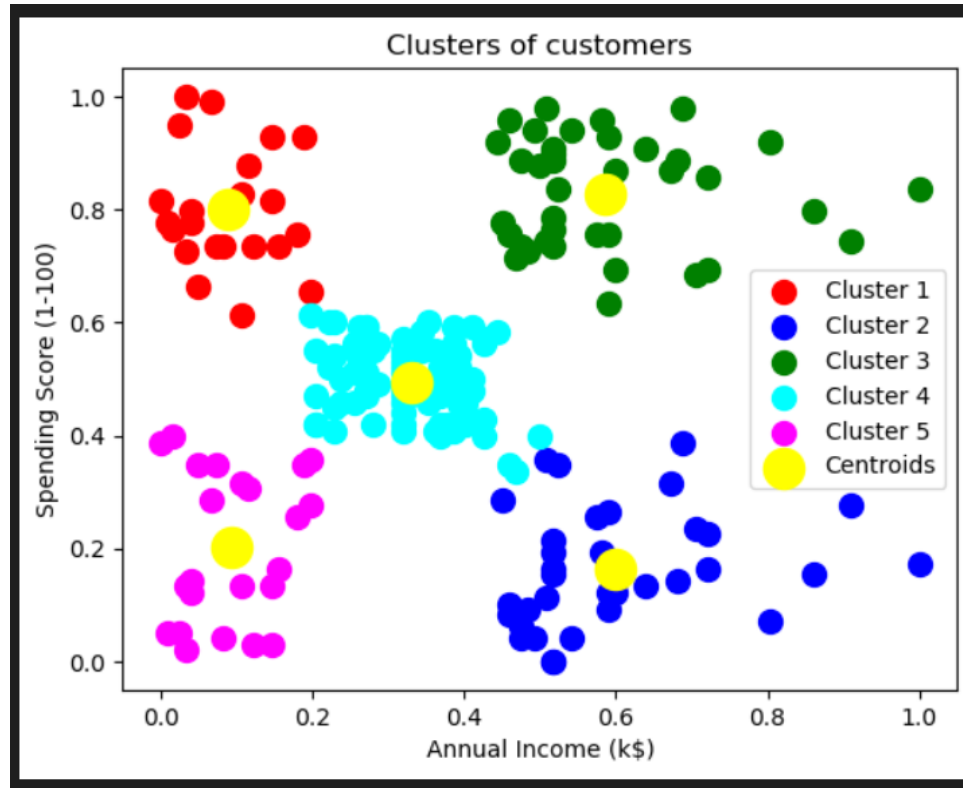
4.Legend: `plt.legend()` displays a legend with labels for each cluster and the centroids.

5.Show the Plot: `plt.show()` renders the plot.

Important Note: If you have only 4 clusters, the line `plt.scatter(X[y_kmeans == 4, 0], X[y_kmeans == 4, 1], s = 100, c = 'magenta', label = 'Cluster 5')` will cause an error because `y_kmeans` will not have a cluster labeled 4. Be sure to match the number of clusters specified in `n_clusters` to the clusters you're plotting.



K-means Cont.



K-means Cont.

```
[35] ✓ 0.0s
dataset["clusters"]=y_kmeans

[36] ✓ 0.0s
dataset
```

CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)	clusters	
0	1	Male	19	15	39	4
1	2	Male	21	15	81	0
2	3	Female	20	16	6	4
3	4	Female	23	16	77	0
4	5	Female	31	17	40	4
...	...	...	...	...	...	...
195	196	Female	35	120	79	2
196	197	Female	45	126	28	1
197	198	Male	32	126	74	2
198	199	Male	32	137	18	1
199	200	Male	30	137	83	2

200 rows × 6 columns



merit.samir@nmu.edu.eg

Thank you

